

Universidad de San Carlos de Guatemala
Facultad de Ingeniería
Ingeniería en Ciencias y Sistemas

CURSO: Arquitectura de Computadores y Ensambladores 1

PROYECTO: Invernadero Inteligente IoT

GRUPO: ARQUI1V1S_G8

INFORME TÉCNICO

Proyecto de Invernadero Inteligente IoT



Integrantes:

1. Mario Rodrigo Balam Churumel
2. Arelis Esther Orozco Hernández
3. Jennyfer Melisa Chicoj Perez
4. Jazmin Rocio Lopez Lima
5. Adilzon Alfredo Velásquez Hernández

Carné No.:

202200147
202300465
202406701
202243063
201908076

Catedrático: LUIS FERNANDO ESPINO BARRIOS

Auxiliar: DIEGO JOSUE GUEVARA ABAJ

Guatemala, Domingo 14 Junio 2026

Índice

- 1. Introducción
- 2. Objetivos
- 3. Alcance del proyecto
- 4. Estado actual del sistema
- 5. Arquitectura general
- 6. Estructura del repositorio
- 7. Subsistemas implementados
- 8. Conexiones físicas y pines
- 9. Comunicación MQTT
- 10. Persistencia en MongoDB Atlas
- 11. Dashboard web
- 12. Archivo lecturas.csv
- 13. Procesamiento ARM64/AArch64
- 14. Integración Python - ARM64 - MongoDB - Dashboard
- 15. Pruebas y evidencias
- 16. Conclusiones
- 17. Anexos

1. Introducción

El presente informe técnico documenta el desarrollo, implementación y resultados del proyecto "**Invernadero Inteligente IoT**", correspondiente al curso Arquitectura de Computadores 1, del grupo **ARQUI1V1S_G8**, durante el segundo semestre de 2026.

El sistema implementado consiste en una maqueta física de invernadero funcional, dividida en dos áreas de cultivo y un centro de control, automatizada mediante una **Raspberry Pi 3/4** como unidad central de procesamiento. La solución integra:

- **Sensores ambientales y de suelo:** DHT11/DHT22 (temperatura y humedad ambiental), dos sensores de humedad de suelo (uno por área), LDR (nivel de luz) y MQ-2/MQ-135 (detección de gas/humo).
- **Actuadores:** bomba de agua para riego funcional, ventilador DC para extracción de aire, LEDs blancos para iluminación artificial, buzzer y LEDs de estado (verde, amarillo, rojo).
- **Centro de control local:** pantalla LCD 16x2 con interfaz I2C y cuatro botones físicos para operación manual.
- **Comunicación IoT:** protocolo MQTT (MQTTX) para intercambio de datos y comandos entre la Raspberry Pi y el dashboard web.
- **Persistencia en la nube:** MongoDB Atlas para almacenamiento histórico de lecturas, eventos, comandos y resultados de procesamiento.
- **Dashboard web:** interfaz gráfica para monitoreo en tiempo real, visualización de gráficas históricas, control remoto de actuadores y visualización de resultados estadísticos.
- **Procesamiento de bajo nivel:** cinco módulos independientes en lenguaje ensamblador **ARM64/AArch64** que calculan estadísticas y tendencias sobre 30 lecturas reales del sistema.

La documentación fue elaborada con base en el programa actualizado entregado en el repositorio oficial del proyecto. En las secciones donde la evidencia física o los resultados de ejecución se encuentran en proceso de integración final, se han dejado campos claramente identificados para completar con capturas de pantalla, fotografías de la maqueta, registros de depuración con GDB y archivos de salida de los módulos ARM64.

El objetivo de este informe es demostrar el cumplimiento del **objetivo SMART** definido en el enunciado, así como la correcta integración de hardware, software, comunicación IoT, persistencia de datos y procesamiento en ensamblador, cumpliendo con todos los criterios establecidos en la rúbrica de calificación.

2. Objetivos

2.1 Objetivo general

Diseñar e implementar un sistema IoT para automatizar el monitoreo y control de un invernadero dividido en dos áreas de cultivo y un centro de control, utilizando Raspberry Pi, Python, MQTT, MongoDB Atlas, dashboard web y análisis estadístico en ARM64.

2.2 Objetivos específicos

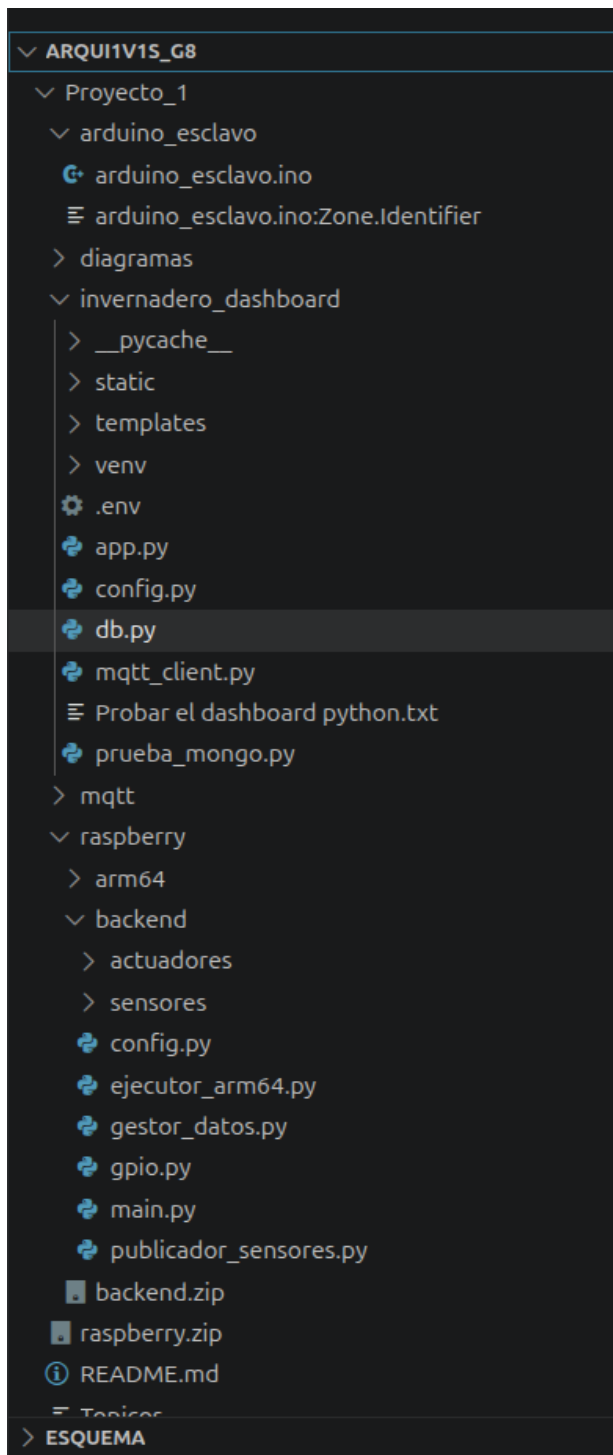
- Medir temperatura, humedad ambiental, humedad de suelo, luz y gas mediante sensores conectados al sistema.
- Controlar riego, ventilación, luces y alarma mediante actuadores físicos o simulados durante pruebas.
- Publicar lecturas, estados y comandos por MQTT usando la base de topics oficial del grupo.
- Persistir lecturas, eventos, comandos, estados y resultados ARM64 en MongoDB Atlas.
- Visualizar el sistema desde un dashboard web con panel principal, gráficas, controles, historial y sección de análisis ARM64.
- Generar un archivo lecturas.csv con 30 registros y procesarlo mediante módulos ARM64/AArch64.

3. Alcance del proyecto

El proyecto contempla la construcción de una maqueta de invernadero con dos áreas de cultivo y un centro de control. El sistema debe demostrar lectura de sensores, actuación automática/manual, comunicación IoT, almacenamiento de datos, visualización web y procesamiento estadístico en bajo nivel.

Elemento	Alcance documentado
Maqueta física	Debe mostrar dos áreas de cultivo y centro de control.
Control principal	Raspberry Pi como unidad principal. Arduino UNO se usa como apoyo para señales analógicas.
Backend	Flask, Flask-SocketIO, MQTT y MongoDB Atlas.
Dashboard	HTML, CSS, JavaScript, Chart.js y Socket.IO.
Sensores	DHT11, humedad de suelo, MQ135; LDR
Actuadores	Gas/alarma, ventilador y riego. Luces y LCD/botones
ARM64	utils.s, modulo_2_varianza.s, modulo_3_anomalias.s y Makefile; faltan módulos 1, 4 y 5 en el ZIP actualizado.



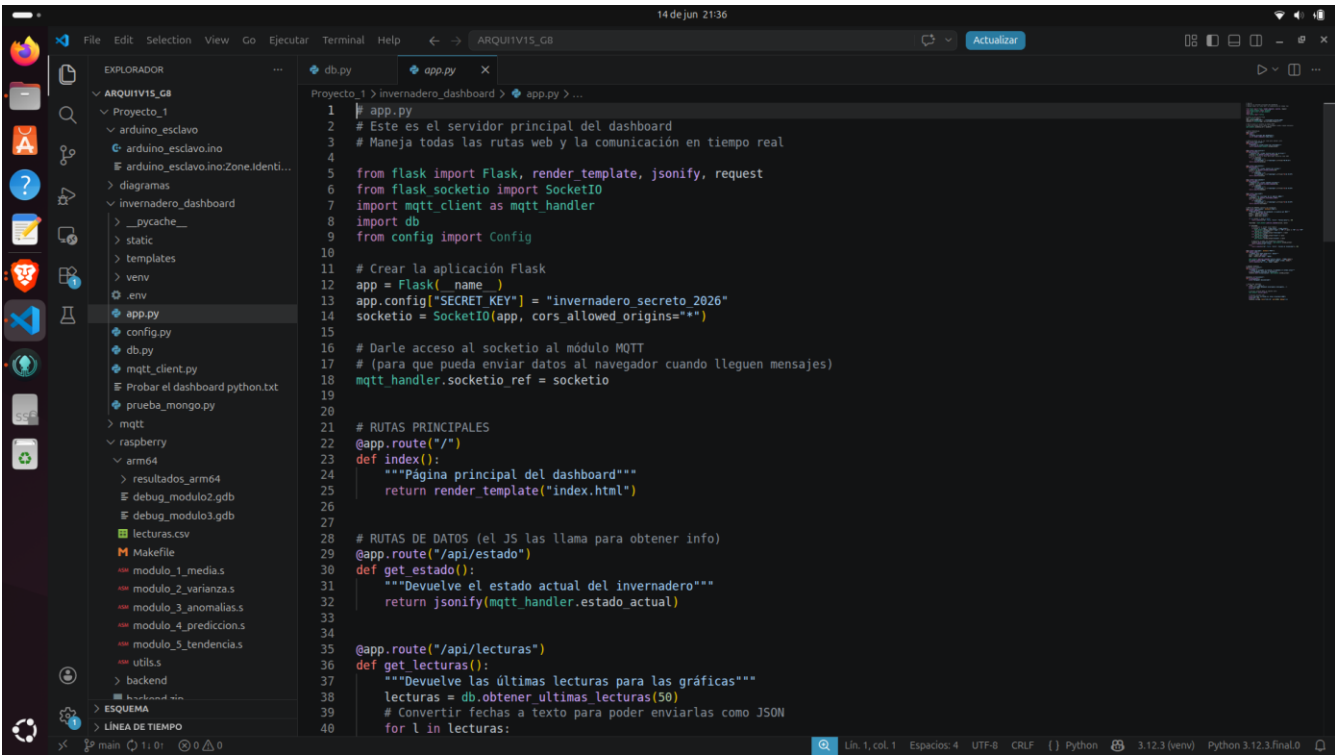


- ▼ ARQUI1V1S_G8
 - ▼ Proyecto_1
 - ▼ raspberry
 - ▼ arm64
 - > resultados_arm64
 - ≡ debug_modulo2.gdb
 - ≡ debug_modulo3.gdb
 - lecturas.csv
 - Makefile
 - ASM modulo_1_media.s
 - ASM modulo_2_varianza.s
 - ASM modulo_3_anomalias.s
 - ASM modulo_4_prediccion.s
 - ASM modulo_5_tendencia.s
 - ASM utils.s
 - ▼ backend
 - ▼ actuadores
 - actuator_luz.py
 - actuator_ventilador.py
 - botones_fisicos.py
 - lcd.py
 - riego.py
 - > sensores
 - config.py
 - ejecutor_arm64.py
 - gestor_datos.py
 - gpio.py
 - main.py
 - publicador_sensores.py
 - backend.zip
 - raspberry.zip
 - ① README.md
 - ≡ Topics
- > ESQUEMA

4. Estado actual del sistema

El código actualizado contiene una estructura funcional inicial para dashboard, MQTT, MongoDB y algunos módulos de sensores, actuadores y ARM64. La siguiente tabla resume el estado observado del proyecto en los archivos entregados.

Componente	Estado actual	Evidencia en código
Dashboard web	Avanzado / funcional para interfaz	invernadero_dashboard/app.py, templates/index.html, static/dashboard.js
MQTT dashboard	Configurado con base topic oficial	invernadero_dashboard/config.py y mqtt_client.py
MongoDB Atlas	Configurado con base de datos P1_G8_ARQUI1 y colecciones requeridas	invernadero_dashboard/db.py
Sensor DHT11	Implementado, pero topics deben alinearse a base oficial	raspberr/sensores/sensor_dht11.py
Sensor suelo	Implementado por lectura serial Arduino; topics deben alinearse a base oficial	raspberr/sensores/sensor_suelo.py
Sensor MQ135	Implementado con modo simulación y publicación MQTT oficial	raspberr/sensores/sensor_mq135.py
Actuador gas/alarma	Implementado con buzzer y LEDs de estado	raspberr/actuadores/actuador_gas.py
Actuador ventilador	Implementado con temperatura/gas; topic temperatura debe alinearse	raspberr/actuadores/actuador_ventilador.py
Actuador riego	Implementado con invernadero/G8/P1/actuadores/riego_area1 invernadero/G8/P1/actuadores/riego_area2	raspberr/actuadores/actuador_riego.py
ARM64	Parcial: módulos 2 y 3 presentes; Makefile espera cinco módulos	raspberr/arm64/
lecturas.csv	Existe con 30 registros de prueba	raspberr/arm64/lecturas.csv



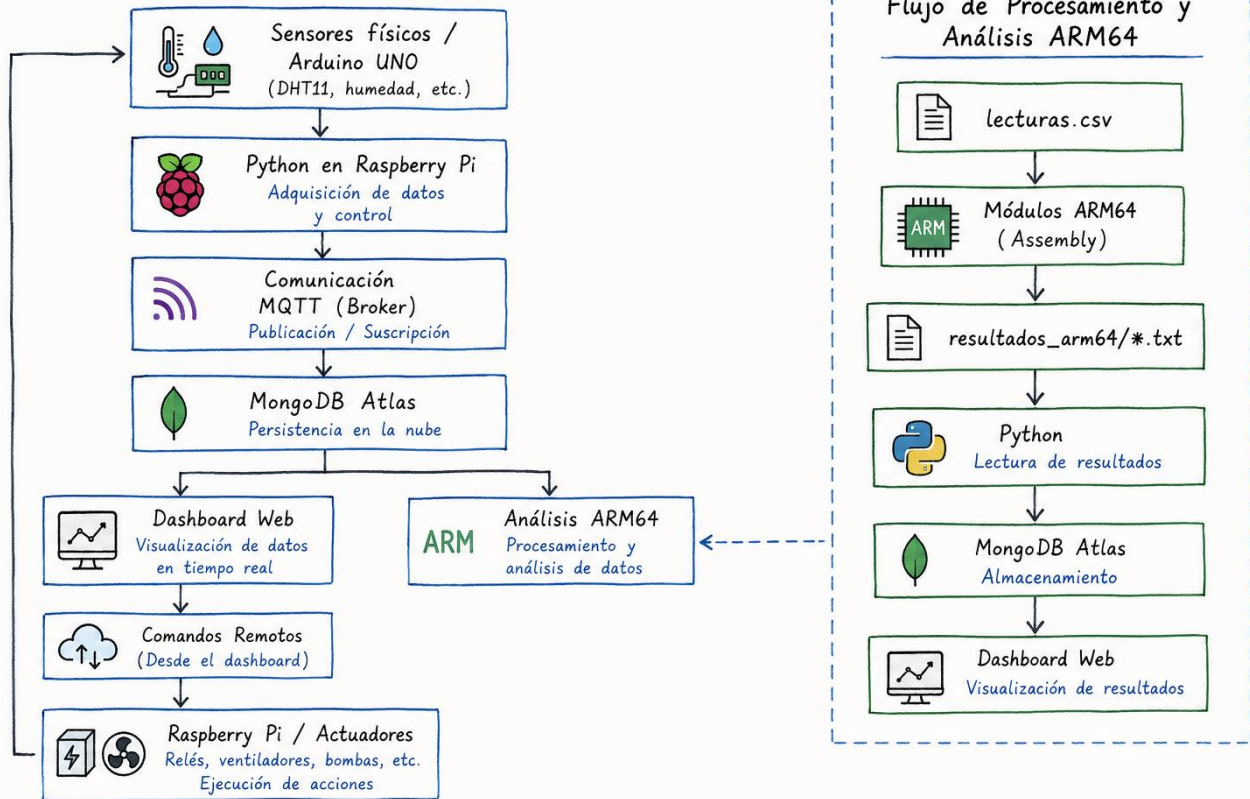
The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows the project structure, including folders like 'invernadero_dashboard' and 'raspberr', and files like 'app.py', 'config.py', 'db.py', 'mqtt_client.py', 'prueba_mongo.py', 'mqtt', 'arm64', 'resultados_arm64', 'debug_modulo2.gdb', 'debug_modulo3.gdb', 'lecturas.csv', 'Makefile', 'modulo_1_medias.s', 'modulo_2_varianza.s', 'modulo_3_anomalias.s', 'modulo_4_prediccion.s', 'modulo_5_tendencias.s', 'utils.s', 'backends', 'ESQUEMA', and 'LINEA DE TIEMPO'. The code editor shows the main 'app.py' file, which is a Flask application. The code includes imports for Flask, SocketIO, and MQTT, and defines routes for the dashboard and data endpoints. The code is as follows:

```
1 app.py
2 # Este es el servidor principal del dashboard
3 # Maneja todas las rutas web y la comunicación en tiempo real
4
5 from flask import Flask, render_template, jsonify, request
6 from flask_socketio import SocketIO
7 import mqtt_client as mqtt_handler
8 import db
9 from config import Config
10
11 # Crear la aplicación Flask
12 app = Flask(__name__)
13 app.config["SECRET_KEY"] = "invernadero secreto 2026"
14 socketio = SocketIO(app, cors_allowed_origins="")
15
16 # Darle acceso al socketio al módulo MQTT
17 # (para que pueda enviar datos al navegador cuando lleguen mensajes)
18 mqtt_handler.socketio_ref = socketio
19
20 # RUTAS PRINCIPALES
21
22 @app.route("/")
23 def index():
24     """Página principal del dashboard"""
25     return render_template("index.html")
26
27 # RUTAS DE DATOS (el JS las llama para obtener info)
28 @app.route("/api/estado")
29 def get_estado():
30     """Devuelve el estado actual del invernadero"""
31     return jsonify(mqtt_handler.estado_actual)
32
33 @app.route("/api/lecturas")
34 def get_lecturas():
35     """Devuelve las últimas lecturas para las gráficas"""
36     lecturas = db.obtener_ultimas_lecturas(50)
37     # Convertir fechas a texto para poder enviarlas como JSON
38     for l in lecturas:
```

5. Arquitectura general

La arquitectura está organizada en capas: adquisición de datos, control en Python, comunicación MQTT, persistencia en MongoDB, visualización web y análisis ARM64.

Arquitectura General del Sistema IoT



6. Estructura del repositorio

```

ARQUI1V1S_G8/
├── Proyecto_1/
│   ├── README.md
│   ├── Topics
│   ├── diagramas/
│   ├── invernadero_dashboard/
│   │   ├── app.py
│   │   ├── config.py
│   │   ├── db.py
│   │   ├── mqtt_client.py
│   │   ├── templates/index.html
│   │   └── static/
│   ├── mqtt/
│   ├── raspberry/
│   │   ├── sensores/
│   │   ├── actuadores/
│   │   └── arm64/
  
```

Carpeta / archivo	Función
invernadero_dashboard/app.py	Servidor Flask principal y rutas API.
invernadero_dashboard/config.py	Configuración de broker MQTT, MongoDB y topics oficiales.
invernadero_dashboard/db.py	Conexión y operaciones sobre MongoDB Atlas.
invernadero_dashboard/mqtt_client.py	Suscripción MQTT, actualización de estado y envío de comandos.
templates/index.html	Estructura visual del dashboard.

static/dashboard.js	Lógica de frontend, Socket.IO, gráficas, botones y actualización de panel.
raspberr/sensores/	Scripts de sensores DHT11, humedad de suelo y MQ135.
raspberr/actuadores/	Scripts de actuadores de gas/alarma, riego y ventilador.
raspberr/arm64/	Módulos ensamblador ARM64, utils.s, Makefile y lecturas.csv.
Topicos	Listado oficial de topics MQTT del proyecto.

7. Subsistemas implementados

7.1 Monitoreo ambiental con DHT11

El script sensor_dht11.py lee temperatura y humedad ambiental mediante un sensor DHT11 conectado en board.D4/GPIO4. Publica lecturas en formato JSON con fecha, valor y estado clasificado.

Parámetro	Valor observado
Archivo	raspberr/sensores/sensor_dht11.py
Sensor	DHT11
Pin indicado en código	board.D4 / GPIO4
Broker	broker.emqx.io
Puerto	1883
Cliente MQTT	sensor_dht11_G8
Clasificación temperatura	BAJA < 20; NORMAL 20 a 30; ALTA > 30
Clasificación humedad	BAJA < 30%; NORMAL 30% a 80%; ALTA > 80%
Nota de integración	Actualizar TOPIC_TEMP y TOPIC_HUM para usar invernadero/G8/P1/sensores/temperatura e invernadero/G8/P1/sensores/humedad_ambiente.

7.2 Monitoreo de humedad de suelo por área

El script sensor_suelo.py lee dos valores enviados por Arduino UNO mediante puerto serial. Cada línea esperada tiene el formato valor_area1,valor_area2. Cada lectura se clasifica como SECO, NORMAL o SATURADO.

Parámetro	Valor observado
Archivo	raspberr/sensores/sensor_suelo.py
Puerto serial	/dev/ttyUSB0
Velocidad	9600 baudios
Cliente MQTT	sensor_suelo_G9
Umbral seco	valor >= 700
Umbral saturado	valor <= 300
Estado normal	301 a 699
Frecuencia de lectura	cada 10 segundos
Nota de integración	Actualizar TOPIC_1 y TOPIC_2 para usar la base invernadero/G8/P1.

7.3 Detección de gas con MQ135

El script sensor_mq135.py implementa monitoreo de gas/calidad de aire. Puede trabajar en simulación si no se encuentran las librerías del ADS1115. Calcula una línea base, umbral de advertencia y umbral de emergencia.

Parámetro	Valor observado
Archivo	raspberr/sensores/sensor_mq135.py
Sensor	MQ135
Canal ADC	0 mediante ADS1115
Base topic	invernadero/G8/P1
Topic gas	invernadero/G8/P1/sensores/gas
Topic eventos	invernadero/G8/P1/eventos
Cliente MQTT	sensor_mq135_G8
Modo simulación	Activo por defecto en el código entregado
Factor advertencia	1.25
Factor emergencia	1.60
Separación mínima	1500
Muestras promedio	8

Tiempo de ciclo	2 segundos
-----------------	------------

7.4 Alarma y estado global por gas

El script `actuador_gas.py` suscribe al topic del sensor MQ135 y controla buzzer y LEDs de estado. Convierte GAS_NORMAL, GAS_ADVERTENCIA y GAS_EMERGENCIA en estado global NORMAL, ADVERTENCIA o EMERGENCIA.

Elemento	Detalle
Archivo	<code>raspberrypi/actuadores/actuador_gas.py</code>
Buzzer	GPIO23
LED verde	GPIO17
LED amarillo	GPIO27
LED rojo	GPIO22
Topic gas	<code>invernadero/G8/P1/sensores/gas</code>
Topic alarma	<code>invernadero/G8/P1/actuadores/alarma</code>
Topic estado global	<code>invernadero/G8/P1/estado/global</code>
Topic control manual	<code>invernadero/G8/P1/control/manual</code>
Comandos manuales	SILENCIAR/OFF y REACTIVAR/ON para alarma

7.5 Ventilación

El script `actuador_ventilador.py` controla un ventilador en GPIO18. El ventilador se enciende si la temperatura supera 30 °C o si el estado de gas es GAS_ADVERTENCIA o GAS_EMERGENCIA.

Elemento	Detalle
Archivo	<code>raspberrypi/actuadores/actuador_ventilador.py</code>
Pin ventilador	GPIO18
Umbral temperatura	30 °C
Topic gas	<code>invernadero/G8/P1/sensores/gas</code>
Topic ventilador	<code>invernadero/G8/P1/actuadores/ventilador</code>
Nota de integración	Actualizar TOPIC_TEMP a <code>invernadero/G8/P1/sensores/temperatura</code> para coincidir con los topics oficiales.

7.6 Riego funcional

El archivo `actuador_riego.py` contiene la lógica para riego por área, duración controlada y pausa mínima entre ciclos. El diseño esperado es activar el relé de riego cuando el sensor de suelo indique SECO y bloquearlo cuando indique SATURADO.

Elemento	Detalle
Archivo	<code>raspberrypi/actuadores/actuador_riego.py</code>
Duración de riego	5 segundos
Pausa mínima	30 segundos
Pin área 1	GPIO17 según código actual
Pin área 2	GPIO27 según Código actual
Estado del archivo	Requiere corrección de PORT, PIN_RIEGO_2, import datetime, nombres CLIENTE_ID/CLIENT_ID, indentación de callbacks y topics oficiales.

7.7 Iluminación, LCD y botones físicos

El sistema de iluminación inteligente, la pantalla LCD y los botones físicos forman parte fundamental del centro de control local del invernadero. A continuación se detalla su implementación.

Elemento	Estado en documentación
Sensor LDR / KY-018	implementado. Se conecta a un pin analógico mediante un divisor de tensión o a un ADC MCP3008 (canal 0). Se lee cada 2 segundos y se publica en el topic MQTT <code>invernadero/sensores/luz</code> . Umbral configurado: < 300 = oscuro (enciende luces), ≥ 300 = suficiente luz (apaga luces). Archivo implementado

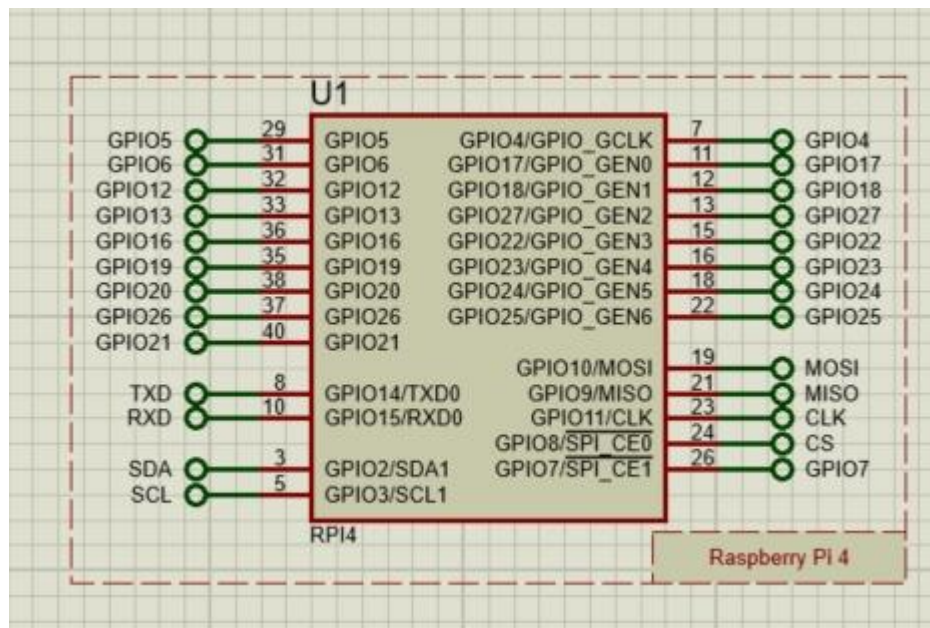
Luces blancas por área	Implementado. Dos LEDs blancos de alta luminosidad, uno en cada área de cultivo. Conectados a los pines GPIO 17 (Área 1) y GPIO 27 (Área 2) mediante resistencias de 220Ω. Control automático por LDR y control manual desde botones físicos o dashboard. Archivo:
LCD 16x2 o 20x4	Implementado. Pantalla LCD 16x2 con interfaz I2C (dirección 0x27). Muestra información rotativa cada 3 segundos: temperatura, humedad ambiental, humedad de suelo por área, nivel de luz, nivel de gas, estado de riego, ventilación, luces y estado global del invernadero. En emergencia por gas, muestra mensaje fijo de alerta. Archivo
Botones físicos	Implementado. Cuatro botones pulsadores conectados a pines GPIO 5, 6, 13 y 19 con resistencia pull-up interna activada. Funciones: Botón 1 (GPIO 5) → Cambiar modo Auto/Manual. Botón 2 (GPIO 6) → Activar riego manual. Botón 3 (GPIO 13) → Encender/apagar luces manualmente. Botón 4 (GPIO 19) → Silenciar buzzer. Con detección por interrupción (falling edge) y debounce de 300ms. Archivo
Control remoto de luces	Implementado. El dashboard web envía comandos MQTT al topic invernadero/control/remoto con payload {"luces": "ON"}, {"luces": "OFF"} o {"modo_luces": "AUTO"}. La Raspberry Pi suscrita al topic recibe y ejecuta los comandos actualizando los LEDs y el estado del sistema. También se visualiza en tiempo real el estado de las luces en el dashboard

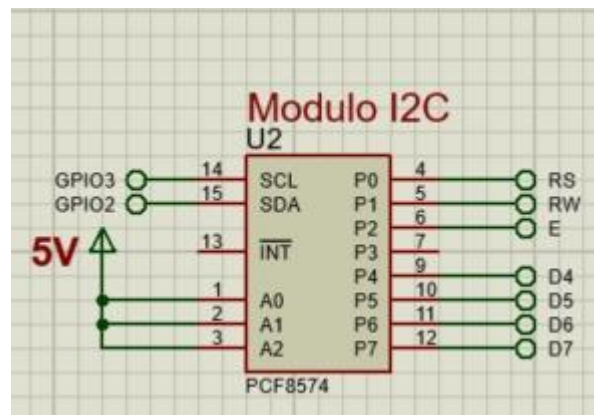
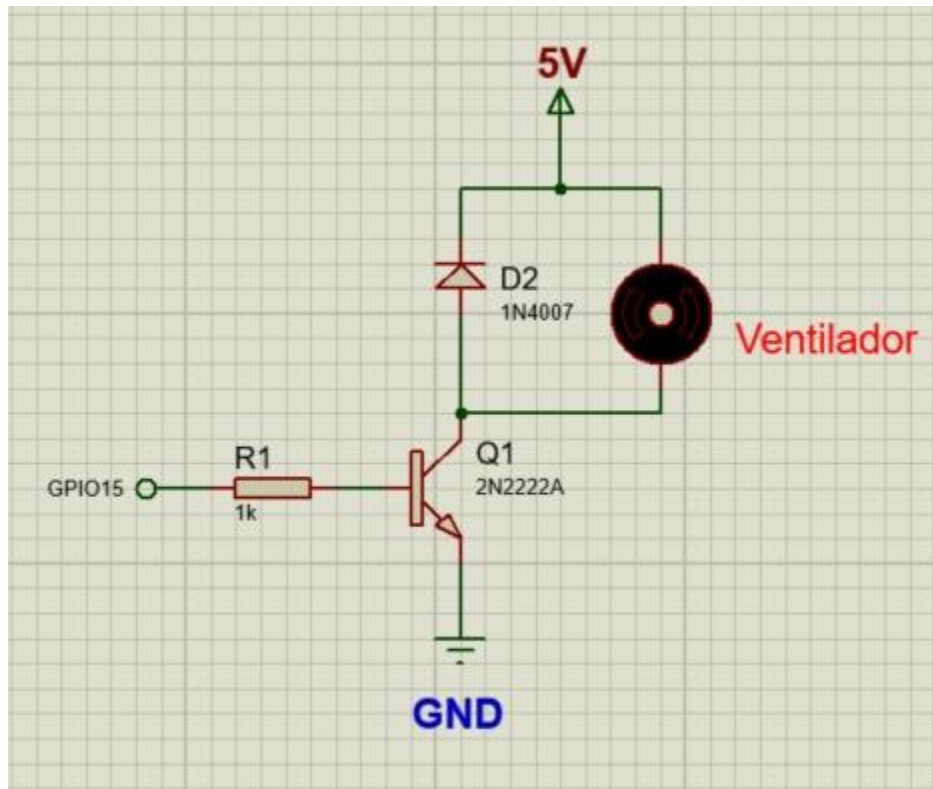
S

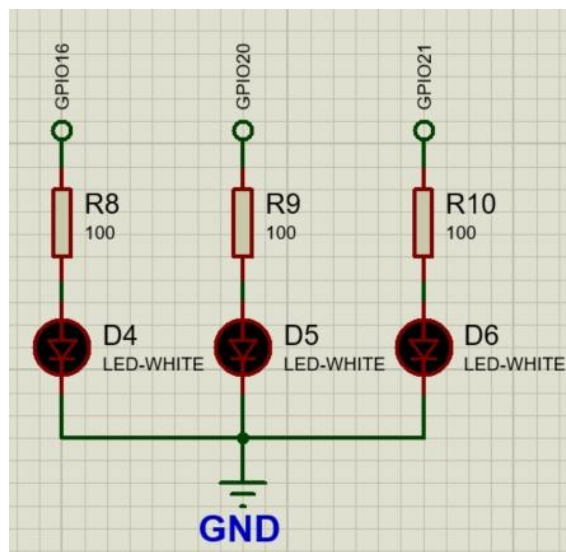
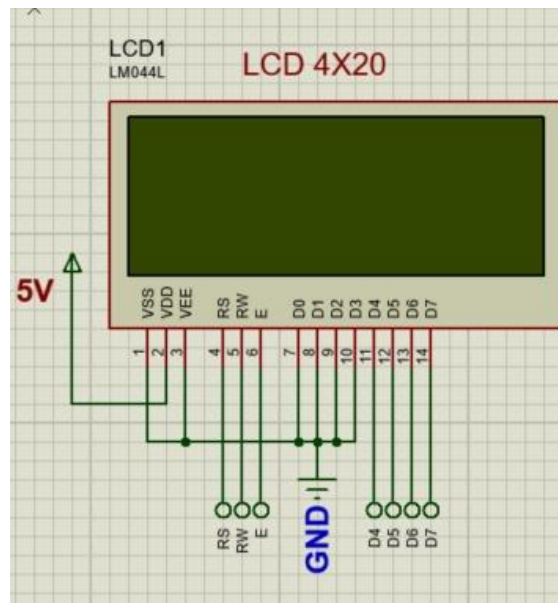
8. Conexiones físicas y asignación de pines

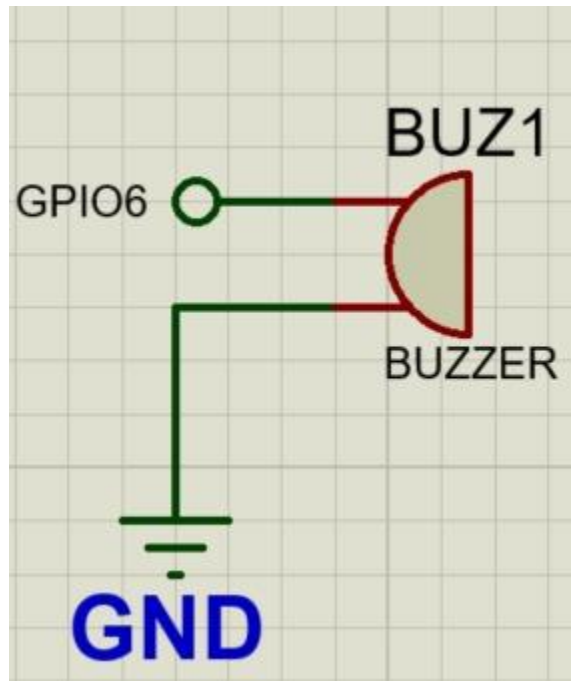
La tabla queda preparada para completar durante la integración física final. Se incluyen únicamente pines observados en el código actualizado; los demás quedan como pendiente para no inventar datos.

Componente	Pin observado / pendiente	Alimentación	Tipo	Observación
DHT11	GPIO4 / board.D4	3.3V o 5V	Digital	Lectura de temperatura y humedad.
Humedad suelo área 1	Pendiente en Arduino	5V	Analógica	Lectura serial desde Arduino.
Humedad suelo área 2	Pendiente en Arduino	5V	Analógica	Lectura serial desde Arduino.
LDR / KY-018	Pendiente	5V	Analógica	Pendiente de archivo/evidencia.
MQ135	ADS1115 canal 0	5V	Analógica	Calibración por línea base.
Bomba / relé riego área 1	GPIO17 según archivo actual	Fuente externa recomendada	Digital	Revisar conflicto con LED verde si se usan ambos.
Bomba / relé riego área 2	Pendiente	Fuente externa recomendada	Digital	PIN_RIEGO_2 debe definirse.
Ventilador	GPIO18	Fuente externa recomendada	Digital	Activación por temperatura o gas.
Buzzer	GPIO23	GPIO / módulo	Digital	Alarma sonora.
LED verde	GPIO17	GPIO	Digital	Estado NORMAL; revisar conflicto con riego área 1.
LED amarillo	GPIO27	GPIO	Digital	Estado ADVERTENCIA.
LED rojo	GPIO22	GPIO	Digital	Estado EMERGENCIA.
LCD I2C	Pendiente SDA/SCL	3.3V o 5V	I2C	Pendiente de integración.
Botones físicos	Pendiente	GPIO	Digital	Modo, riego, luces y silencio/reset.









9. Comunicación MQTT

El sistema utiliza MQTT para publicar lecturas, estados y comandos. La configuración oficial se encuentra centralizada en `invernadero_dashboard/config.py` con la base topic `invernadero/G8/P1`.

Parámetro	Valor
Broker	broker.emqx.io
Puerto	1883
Base topic	invernadero/G8/P1
Usuario/clave	No requerido para broker público. No documentar credenciales reales en el informe.

9.1 Topics oficiales

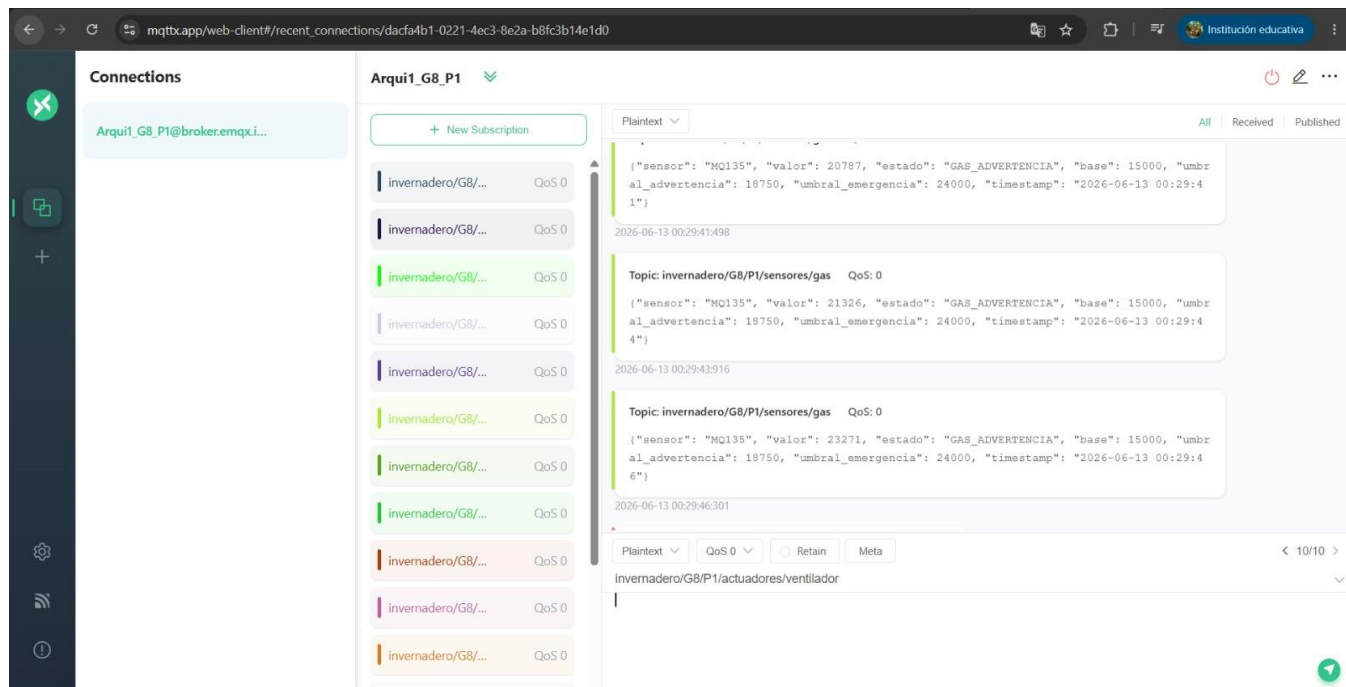
Categoría	Topic	Uso
Sensor	invernadero/G8/P1/sensores/temperatura	Temperatura ambiental
Sensor	invernadero/G8/P1/sensores/humedad_ambiente	Humedad ambiental
Sensor	invernadero/G8/P1/sensores/humedad_suelo_area1	Humedad suelo área 1
Sensor	invernadero/G8/P1/sensores/humedad_suelo_area2	Humedad suelo área 2
Sensor	invernadero/G8/P1/sensores/luz	Nivel de luz
Sensor	invernadero/G8/P1/sensores/gas	Nivel de gas/calidad de aire
Actuador	invernadero/G8/P1/actuadores/riego	Estado general del riego
Actuador	invernadero/G8/P1/actuadores/riego_area1	Riego área 1
Actuador	invernadero/G8/P1/actuadores/riego_area2	Riego área 2
Actuador	invernadero/G8/P1/actuadores/ventilador	Estado/control de ventilador
Actuador	invernadero/G8/P1/actuadores/luces	Estado/control de luces
Actuador	invernadero/G8/P1/actuadores/alarma	Estado/control de alarma
Estado	invernadero/G8/P1/estado/global	Estado global
Control	invernadero/G8/P1/control/remoto	Comandos desde dashboard
Control	invernadero/G8/P1/control/manual	Comandos desde botones físicos

9.2 Flujo MQTT

Python publica lecturas → Broker MQTT → Dashboard recibe actualización

Dashboard envía comando → Broker MQTT → Raspberry Pi ejecuta actuador

Raspberry Pi publica estado → Broker MQTT → Dashboard y MongoDB actualizan historial



10. Persistencia en MongoDB Atlas

La persistencia se implementa en `invernadero_dashboard/db.py` mediante `pymongo`. La base configurada es `P1_G8_ARQUI1`. Por seguridad, la URI real de MongoDB no debe exponerse en el informe público ni en capturas.

Colección	Propósito	Campos observados / esperados
sensor_readings	Lecturas de sensores	timestamp, tipo, valor, origen, estado
events	Alertas y eventos	timestamp, tipo, descripcion, nivel, origen
commands	Comandos enviados	timestamp, accion, valor, origen
system_status	Estado global actual	timestamp, estado
actuator_logs	Historial de actuadores	Pendiente de uso directo en código actual; reservar para bomba, ventilador, luces, buzzer y LEDs.
arm64_results	Resultados ARM64	timestamp y resultados de módulos ARM64.

10.1 Ejemplos de documentos

sensor_readings:

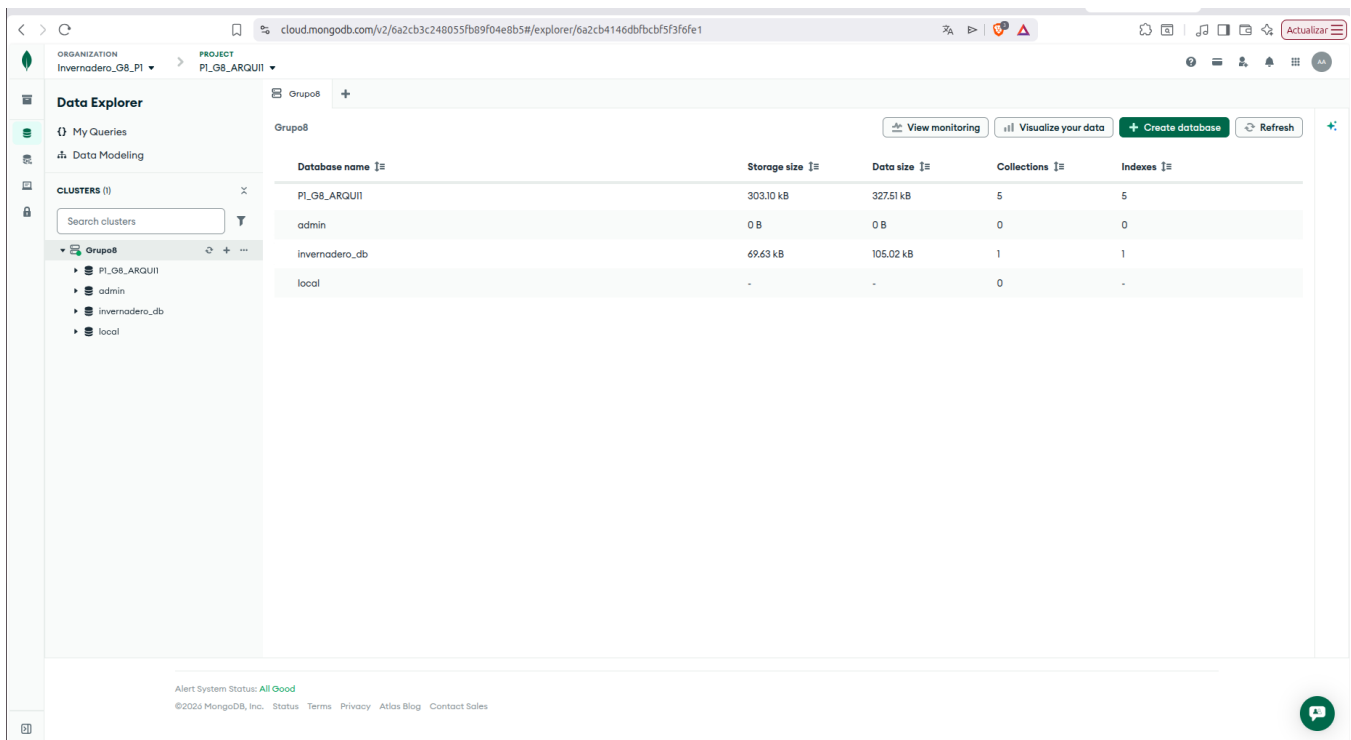
```
{ "timestamp": "fecha_hora", "tipo": "temperatura", "valor": 28, "origen": "sensor", "estado": "activo" }
```

events:

```
{ "timestamp": "fecha_hora", "tipo": "GAS_ALTO", "descripcion": "Nivel de gas: 350", "nivel": "EMERGENCIA", "origen": "dashboard" }
```

commands:

```
{ "timestamp": "fecha_hora", "accion": "ventilador", "valor": "ON", "origen": "dashboard" }
```



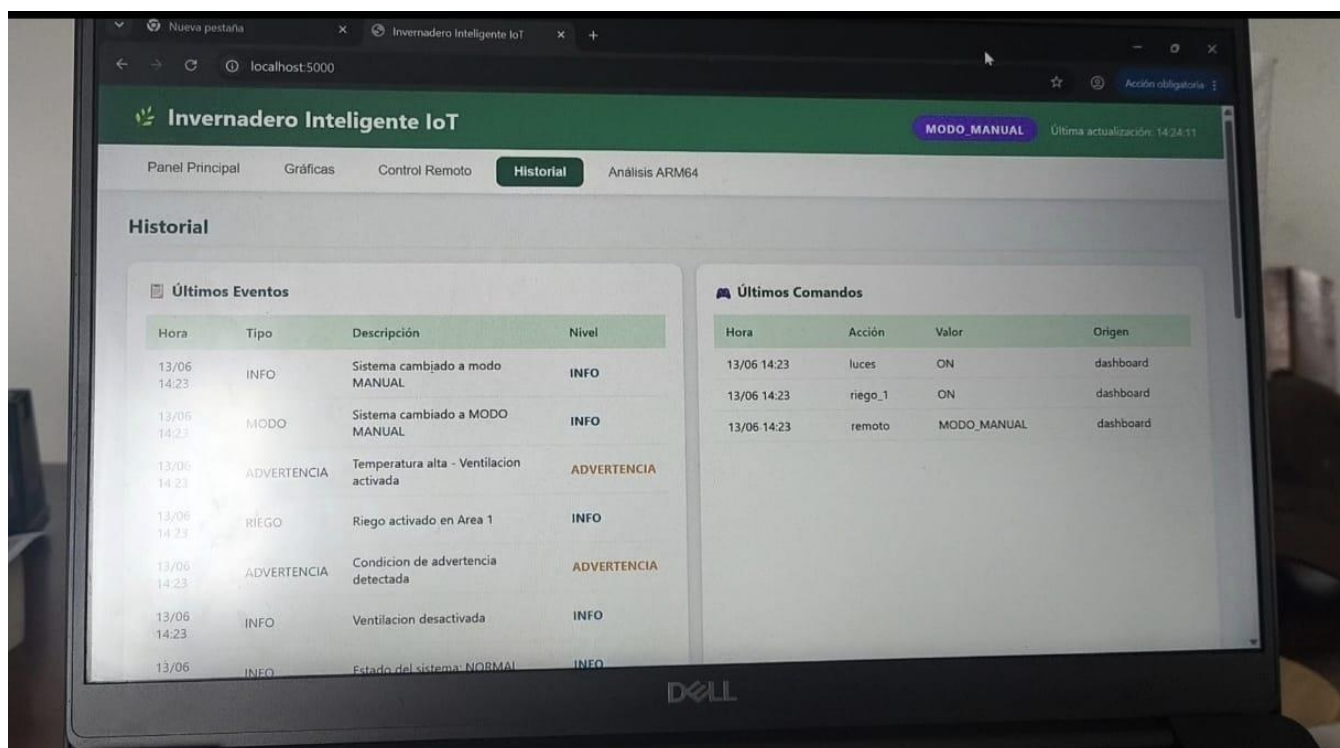
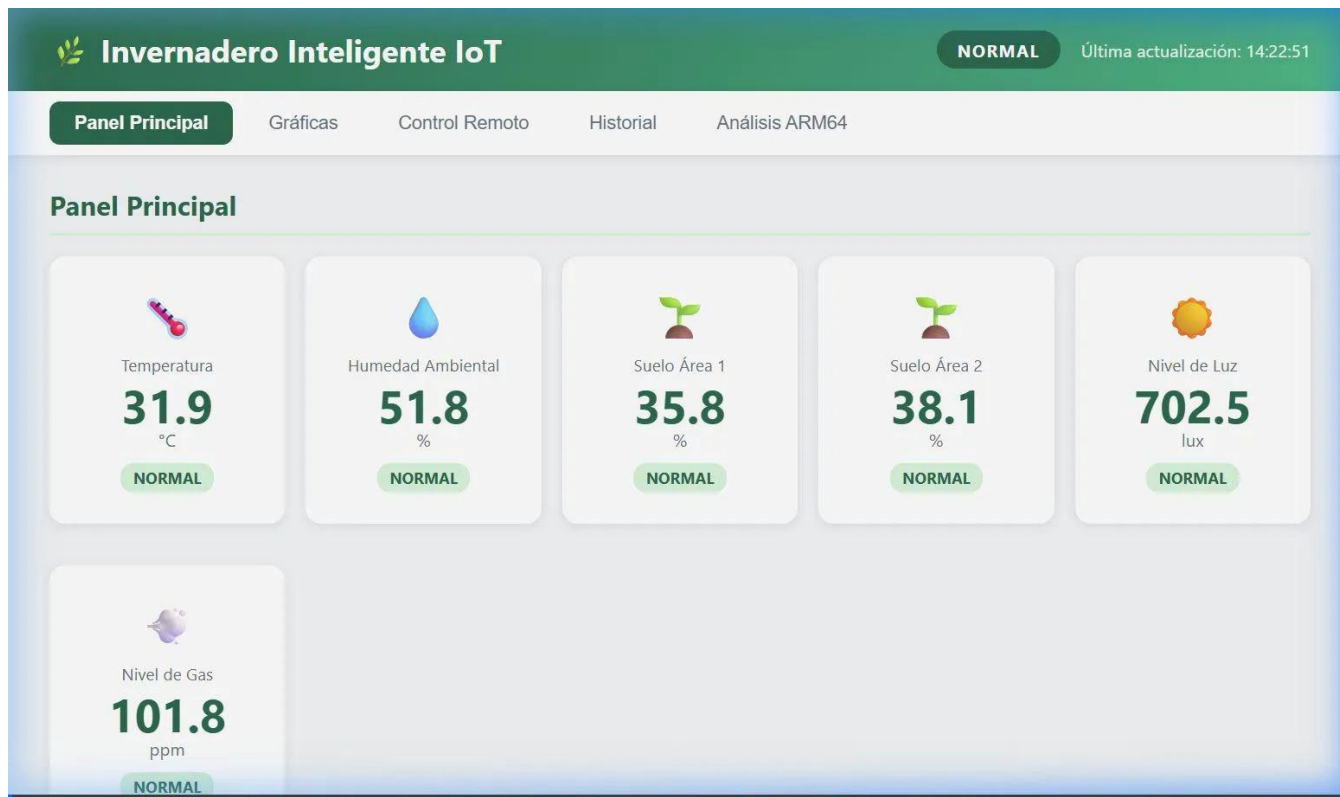
11. Dashboard web

El dashboard web está implementado con Flask, Flask-SocketIO, HTML, CSS y JavaScript. Permite visualizar datos actuales, consultar historial, mostrar gráficas, ejecutar comandos remotos y revisar resultados ARM64.

Archivo	Función
app.py	Define rutas web, APIs /api/estado, /api/lecturas, /api/eventos, /api/comandos, /api/arm64 y comandos POST.
mqtt_client.py	Escucha topics MQTT, actualiza estado_actual, guarda lecturas/eventos y emite Socket.IO.
db.py	Guarda y consulta MongoDB.
templates/index.html	Interfaz con secciones Panel Principal, Gráficas, Control Remoto, Historial y Análisis ARM64.
static/dashboard.js	Actualiza sensores, actuadores, badge global, gráficas e interacción con botones.
static/style.css	Estilos del dashboard, tarjetas, badges, tablas y sección ARM64.

11.1 Funcionalidades visibles

- Panel principal con temperatura, humedad ambiental, humedad de suelo área 1, humedad de suelo área 2, luz y gas.
- Estado de actuadores: riego, ventilador, luces y alarma.
- Gráficas históricas para sensores mediante Chart.js.
- Control remoto para modo, riego, luces, ventilación y alarma.
- Historial de eventos y comandos.
- Sección Análisis ARM64 con selección de variable por módulo.



12. Archivo lecturas.csv

El archivo lecturas.csv ubicado en raspberry/arm64 contiene 30 registros y el encabezado obligatorio para el análisis ARM64.

ID, TEMP, HUM_AIRE, HUM_SUELO_1, HUM_SUELO_2, LUZ, GAS, RIEGO_1, RIEGO_2

Dato	Valor
Cantidad de registros observada	30
Primera lectura	1,28,70,45,48,320,120,0,0
Última lectura	30,28,70,45,48,320,3000,0,0
Observación	Los registros actuales funcionan como base de prueba. Para la entrega final deben provenir de lecturas reales del invernadero físico.

```
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ nano modulo_3_anomalias
.s
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ nano Makefile
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ make clean
rm -rf build
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ make
mkdir -p build
aarch64-linux-gnu-as modulo_3_anomalias.s -o build/modulo_3_anomalias.o
aarch64-linux-gnu-ld build/modulo_3_anomalias.o -o build/modulo_3_anomalias
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ make run
qemu-aarch64 build/modulo_3_anomalias
MODULE=ANOMALY_DETECTION
TOTAL_VALUES=30
MEAN=291
STD_DEV=604
ANOMALIES=2
SYSTEM_RISK=MEDIUM
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ |
```

S

13. Procesamiento ARM64/AArch64

El procesamiento ARM64 se ubica en la carpeta raspberry/arm64. Con la actualización recibida, el informe incorpora la descripción de los cinco módulos individuales: media aritmética ponderada, varianza/desviación estándar, detección de anomalías, predicción lineal simple y tendencia acumulada avanzada. Todos los módulos deben leer lecturas.csv, procesar exactamente 30 datos, recibir la columna por argumento y generar un archivo .txt propio en resultados_arm64/.

Elemento	Estado observado
utils.s	Presente. Incluye rutinas comunes como lectura CSV, impresión, escritura de archivo y conversión entero a ASCII.
Makefile	Presente. Define reglas all, run1..run5, save1..save5 y save_all.
modulo_1_media.s	No encontrado en el ZIP actualizado. El Makefile lo espera.
modulo_2_varianza.s	Presente. Calcula media, varianza y desviación estándar sobre columna definida.
modulo_3_anomalias.s	Presente. Detecta anomalías y clasifica riesgo.
modulo_4_prediccion.s	No encontrado en el ZIP actualizado. El Makefile lo espera.
modulo_5_tendencia.s	No encontrado en el ZIP actualizado. El Makefile lo espera.
resultado_anomalias.txt	Presente. Resultado: ANOMALIES=2, SYSTEM_RISK=MEDIUM.

13.1 Módulos requeridos por integrante

Integrante	Módulo	Archivo	Salida	Estado
Integrante 1	Media aritmética ponderada	modulo_1_media.s	resultado_media.txt	Código completado. Calcula $\sum(X_i * W_i) / \sum W_i$ con pesos 1..30.
Integrante 2	Varianza y desviación estándar	modulo_2_varianza.s	resultado_varianza.txt	Presente. Calcula media, varianza y desviación

				estándar
Integrante 3	Detección estadística de anomalías	modulo_3_anomalias.s	resultado_anomalias.txt	Código proporcionado; validar salida y evidencia GDB.
Integrante 4	Predicción lineal simple	modulo_4_prediccion.s	resultado_prediccion.txt	Código completado. Predice siguiente valor por tendencia promedio.
Integrante 5	Tendencia acumulada avanzada	modulo_5_tendencia.s	resultado_tendencia.txt	Código completado. Analiza rachas y tendencia final UP/DOWN/STABLE.

13.2 Resultado ARM64 observado

```
qemu-aarch64 build/modulo_3_anomalias
MODULE=ANOMALY_DETECTION
TOTAL_VALUES=30
MEAN=291
STD_DEV=604
ANOMALIES=2
SYSTEM_RISK=MEDIUM
```

```
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ nano modulo_3_anomalias
.s
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ nano Makefile
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ make clean
rm -rf build
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ make
mkdir -p build
aarch64-linux-gnu-as modulo_3_anomalias.s -o build/modulo_3_anom
alias.o
aarch64-linux-gnu-ld build/modulo_3_anomalias.o -o build/modulo_
3_anomalias
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ make run
qemu-aarch64 build/modulo_3_anomalias
MODULE=ANOMALY_DETECTION
TOTAL_VALUES=30
MEAN=291
STD_DEV=604
ANOMALIES=2
SYSTEM_RISK=MEDIUM
dell@DESKTOP-3IB7BC3:~/Modulo_anomalias$ |
```

Diagrama lógico resumido para el módulo 4 - predicción lineal simple:

Inicio: start
Leer 30 datos desde lecturas.csv usando read_csv_column
Obtener X_inicial = valores[0] y X_final = valores[29]
Calcular TOTAL_DIFF = X_final - X_inicial
Calcular AVG_CHANGE = TOTAL_DIFF / 29 y NEXT_VALUE = X_final + AVG_CHANGE
Construir salida, imprimir en terminal y guardar resultado_prediccion.txt

13.3 Actualización técnica de módulos ARM64

Los cinco módulos ARM64 trabajan sobre una variable X seleccionada desde lecturas.csv. La columna se recibe como argumento de consola. Si no se envía argumento o el valor no es válido, cada módulo usa la columna por defecto 2, correspondiente a TEMP. Las columnas recomendadas son: 2=TEMP, 3=HUM_AIRE, 4=HUM_SUELO_1, 5=HUM_SUELO_2, 6=LUZ y 7=GAS.

El flujo común de cada módulo es: obtener columna desde argv, llamar a read_csv_column de utils.s, validar que se hayan leído exactamente 30 datos, ejecutar el algoritmo estadístico, construir el texto de salida en outbuf, imprimir en terminal y guardar el resultado mediante write_file.

Comandos de referencia para ejecución:

```
cd Proyecto_1/raspberry/arm64
mkdir -p build resultados_arm64
make
qemu-aarch64 build/modulo_1_media 2
qemu-aarch64 build/modulo_2_varianza 2
qemu-aarch64 build/modulo_3_anomalias 7
qemu-aarch64 build/modulo_4_prediccion 2
qemu-aarch64 build/modulo_5_tendencia 4
```

13.4 Descripción individual de módulos ARM64

Módulo 1: Media aritmética ponderada

Archivo: modulo_1_media.s. Salida: resultados_arm64/resultado_media.txt. Calcula SUM_X, WEIGHT_SUM y WEIGHTED_MEAN. Los pesos aumentan progresivamente desde 1 hasta 30. Fórmula: $MEDIA_PONDERADA = \frac{\sum(X_i * W_i)}{\sum W_i}$.

Registros principales:

- x19: columna seleccionada.
- x20: cantidad de datos leídos.
- x21: puntero al arreglo values.
- x22: contador i.
- x23: SUM_X.
- x24: suma ponderada.
- x25: WEIGHT_SUM.
- x26/x27: valores temporales para dato y peso.

Ciclos, saltos y subrutinas: Ciclo principal: recorre los 30 valores, suma cada X_i , calcula $W_{i=i+1}$, acumula $X_i * W_i$ y acumula la suma de pesos. Al finalizar realiza una división entera para obtener la media ponderada.

Módulo 2: Varianza y desviación estándar

Archivo: modulo_2_varianza.s. Salida: resultados_arm64/resultado_varianza.txt. Calcula MEAN, VARIANCE y STD_DEV. Usa dos recorridos: primero obtiene la media y luego la suma de cuadrados para la varianza. La desviación estándar se obtiene mediante int_sqrt_u64.

Registros principales:

- x19: columna seleccionada.
- x20: N.
- x21: puntero a values.
- x22: contador.
- x23: suma o suma de cuadrados.
- x24: media.
- x25: varianza.
- x26: diferencia y diferencia al cuadrado.

Ciclos, saltos y subrutinas: Ciclos: ciclo_suma_varianza para calcular ΣX ; ciclo_cuadrados para calcular $\Sigma (X - \text{MEDIA})^2$. Subrutina relevante: int_sqrt_u64 para raíz cuadrada entera.

Módulo 3: Detección estadística de anomalías

Archivo: modulo_3_anomalias.s. Salida: resultados_arm64/resultado_anomalias.txt. Calcula MEAN, STD_DEV, ANOMALIES y SYSTEM_RISK. La regla usada es equivalente a $|Z| \geq 2$: $|X - \text{MEDIA}| \geq 2 * \text{STD_DEV}$. Clasifica 0 anomalías como NORMAL, 1 a 3 como MEDIUM y 4 o más como HIGH.

Registros principales:

- x19: columna seleccionada.
- x20: N.
- x21: puntero al arreglo.
- x22: contador.
- x23: suma o contador de anomalías.
- x24: media.
- x25: desviación estándar.
- x26: $2 * \text{STD_DEV}$.
- x27: diferencia absoluta.

Ciclos, saltos y subrutinas: Ciclos: suma/media, varianza, detección de anomalías. Saltos: diferencia_positiva, no_es_anomalia, riesgo_normal, riesgo_medium y riesgo_high.

Módulo 4: Predicción lineal simple

Archivo: modulo_4_prediccion.s. Salida: resultados_arm64/resultado_prediccion.txt. Calcula INITIAL_VALUE, FINAL_VALUE, TOTAL_DIFF, AVG_CHANGE y NEXT_VALUE. Usa escala x100 para imprimir AVG_CHANGE y NEXT_VALUE con dos decimales.

Registros principales:

- x19: columna seleccionada.
- x21: puntero a valores.
- x22: valor inicial.
- x23: valor final.
- x24: diferencia total.
- x25: constantes 100 y 29.
- x26: promedio de cambio escalado.
- x27: predicción escalada o puntero final de outbuf.

Ciclos, saltos y subrutinas: No requiere recorrer todos los datos después de la lectura, porque usa el primer valor y el valor 30. La fórmula aplicada es $\text{DIF} = \text{XFINAL} - \text{XINICIAL}$, $\text{AVG} = \text{DIF} / (N - 1)$ y $\text{NEXT} = \text{XFINAL} + \text{AVG}$.

Módulo 5: Tendencia acumulada avanzada

Archivo: modulo_5_tendencia.s. Salida: resultados_arm64/resultado_tendencia.txt. Calcula INCREMENTS, DECREMENTS, MAX_UP_STREAK, MAX_DOWN_STREAK, ACCUM_DIFF y TREND. Compara pares consecutivos $X_i - X_{(i-1)}$, por lo que con 30 datos analiza 29 diferencias.

Registros principales:

- x19: columna seleccionada.
- x20: N.
- x21: puntero a valores.
- x22: valor anterior.

- x23: incrementos.
- x24: decrementos.
- x25: racha actual de subida.
- x26: racha actual de bajada.
- x27: racha máxima de subida.
- x28: racha máxima de bajada.
- x9: diferencia acumulada.

Ciclos, saltos y subrutinas: Ciclo principal: ciclo_tendencia. Saltos: tendencia_incremento, tendencia_decremento, tendencia_estable, tendencia_final_up y tendencia_final_down. La tendencia final se clasifica como UP, DOWN o STABLE según ACCUM_DIFF.

13.5 Formato de entrada y salida

Entrada común: archivo lecturas.csv con encabezado

ID,TEMP,HUM_AIRE,HUM_SUELO_1,HUM_SUELO_2,LUZ,GAS,RIEGO_1,RIEGO_2 y exactamente 30 registros. Los módulos no deben calcular sobre datos inventados ni sobre más o menos registros.

Salida común: cada módulo genera un archivo .txt propio dentro de resultados_arm64/. Estos archivos son la fuente que Python debe leer para almacenar los resultados en MongoDB Atlas, colección arm64_results, y mostrarlos en el dashboard.

Módulo	Archivo	Salida	Campos que debe generar
Media ponderada	modulo_1_media.s	resultado_media.txt	MODULE, TOTAL_VALUES, SUM_X, WEIGHT_SUM, WEIGHTED_MEAN
Varianza	modulo_2_varianza.s	resultado_varianza.txt	MODULE, TOTAL_VALUES, MEAN, VARIANCE, STD_DEV
Anomalías	modulo_3_anomalias.s	resultado_anomalias.txt	MODULE, TOTAL_VALUES, MEAN, STD_DEV, ANOMALIES, SYSTEM_RISK
Predicción	modulo_4_prediccion.s	resultado_prediccion.txt	MODULE, INITIAL_VALUE, FINAL_VALUE, TOTAL_DIFF, AVG_CHANGE, NEXT_VALUE
Tendencia	modulo_5_tendencia.s	resultado_tendencia.txt	MODULE, TOTAL_VALUES, INCREMENTS, DECREMENTS, MAX_UP_STREAK, MAX_DOWN_STREAK, ACCUM_DIFF, TREND

13.6 Actividades pendientes específicas para ARM64

- Guardar los archivos modulo_1_media.s, modulo_2_varianza.s, modulo_3_anomalias.s, modulo_4_prediccion.s y modulo_5_tendencia.s en Proyecto_1/raspberry/arm64/.
- Confirmar que Makefile compile los cinco módulos enlazándolos con utils.s.
- Ejecutar cada módulo con al menos una columna real del CSV y generar los cinco archivos .txt en resultados_arm64/.
- Actualizar el script Python de integración para ejecutar los cinco binarios, leer los .txt y guardar los documentos en MongoDB Atlas en arm64_results.
- Actualizar el dashboard para mostrar los resultados de los cinco módulos en la sección Análisis ARM64.
- Tomar evidencia GDB por integrante: breakpoint, registros usados, memoria values y variables resultado.
- Realizar commit individual de cada archivo .s por el integrante responsable.

14. Integración Python - ARM64 - MongoDB - Dashboard

El flujo requerido consiste en que Python genere lecturas.csv, ejecute los binarios ARM64, lea los archivos .txt generados y almacene los resultados en MongoDB dentro de arm64_results. El dashboard debe consumir esos resultados en la sección Análisis ARM64.

Sensores → Python → lecturas.csv → Módulos ARM64 → resultados_arm64/*.txt → Python → MongoDB
arm64_results → Dashboard Análisis ARM64

Paso	Responsable	Estado actualizado
Generar lecturas.csv	Python / backend o script de Raspberry	Debe generarse con 30 lecturas reales. Ya existe archivo de prueba; confirmar generación automática final.
Compilar ARM64	Makefile / toolchain AArch64	Debe compilar utils.s y los cinco módulos: media, varianza, anomalías, predicción y tendencia.
Ejecutar módulos	QEMU o Raspberry Pi ARM64	Cada módulo recibe la columna por argumento: qemu-aarch64 build/modulo X N.
Leer resultados .txt	Python	Debe leer los cinco archivos en resultados_arm64/ y convertirlos a estructura JSON/diccionario.
Guardar arm64_results	MongoDB Atlas	Colección definida. Guardar módulo, variable, salida, timestamp y archivo origen.
Mostrar en dashboard	Frontend / API /api/arm64	La sección Análisis ARM64 debe mostrar resultados de los cinco módulos.

15. Pruebas y evidencias

La documentación final debe contener evidencias visuales de funcionamiento. Las siguientes pruebas están definidas para completar el informe antes de la entrega.

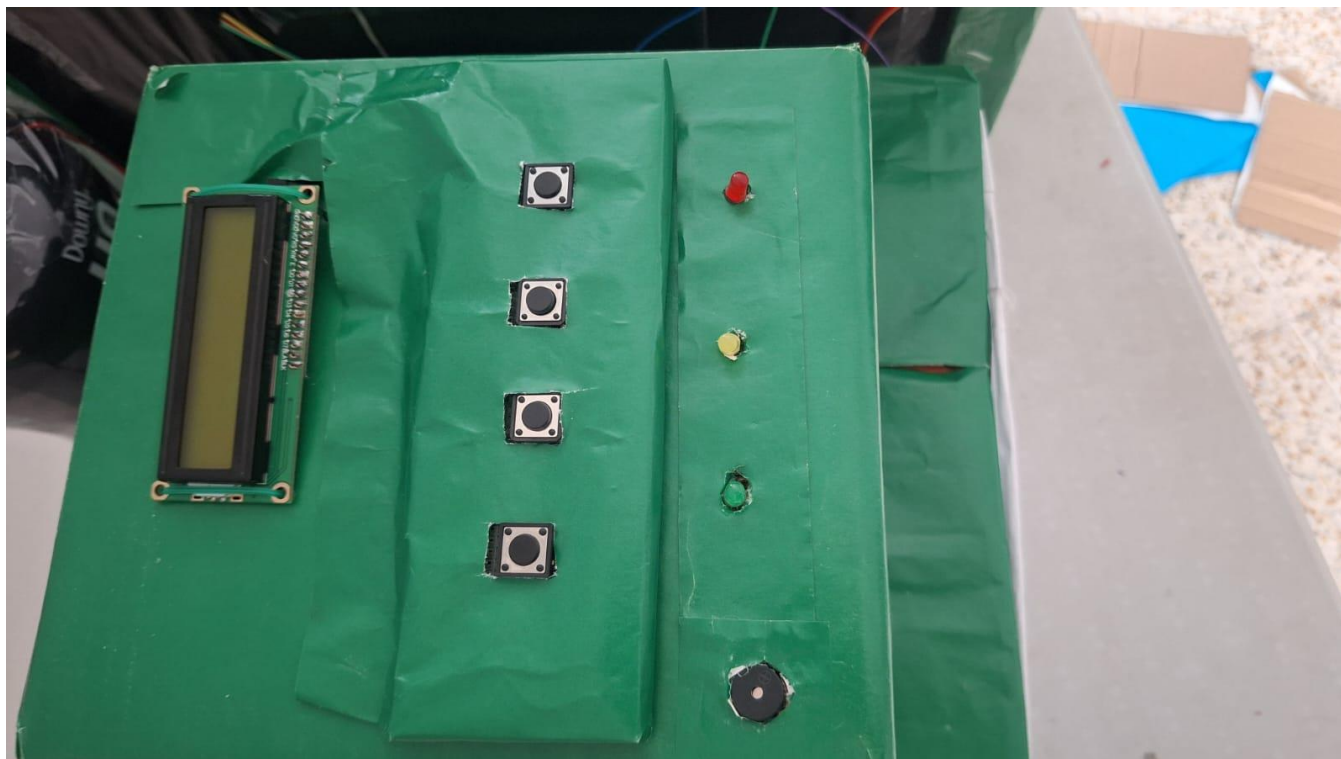
Prueba	Qué debe demostrar	Evidencia a colocar
Dashboard	El servidor Flask abre y muestra panel principal	Captura del navegador
MQTT	Publicación y recepción de topics oficiales	Captura MQTTX o consola
MongoDB	Colecciones con documentos reales	Captura MongoDB Atlas
DHT11	Temperatura y humedad publicadas	Captura consola/MQTTX
Suelo	Lectura de dos áreas por Arduino serial	Captura consola/MQTTX
Gas	Estados GAS NORMAL/ADVERTENCIA/EMERGENCIA	Captura consola/MQTTX
Ventilador	Activación por temperatura alta o gas	Foto/video/captura
Riego	Bomba funcionando con pausa mínima	Video/foto
LCD y botones	Datos rotativos y control local	Foto/video
ARM64	Compilación, ejecución y archivos de salida	Captura terminal y GDB

16. Conclusiones

- El proyecto cuenta con una base funcional para dashboard web, MQTT, MongoDB Atlas y varios scripts de sensores y actuadores.
- La estructura de topics oficiales queda definida bajo la base invernadero/G8/P1, lo que permite organizar lecturas, actuadores, estado global y comandos.
- La sección ARM64 está parcialmente integrada: existe utils.s, Makefile, lecturas.csv y módulos 2 y 3; falta agregar los módulos restantes para cumplir la totalidad del apartado.

- El dashboard ya contempla panel principal, gráficas, control remoto, historial y análisis ARM64, por lo que es una base adecuada para la presentación final.
- Para que el sistema sea defendible en evaluación, deben agregarse evidencias físicas, capturas MQTT/MongoDB/GDB y corregirse los scripts pendientes antes de la entrega.

20. Anexos para completar







Panel Principal



Temperatura

31.9
°C

NORMAL



Humedad Ambiental

51.8
%

NORMAL



Suelo Área 1

35.8
%

NORMAL



Suelo Área 2

38.1
%

NORMAL



Nivel de Luz

702.5
lux

NORMAL



Nivel de Gas

101.8
ppm

NORMAL

Control Remoto

Modo del Sistema

Automático

Manual

Modo actual: AUTO

Control de Riego

Activar Área 1

Activar Área 2

Detener Riego

Control de Luces

Encender

Apagar

Control de Ventilación

Encender

Apagar

Alarma

Silenciar Alarma

Restablecer Estado

cloud.mongodb.com/v2/6a2cb3c248055fb89f04e8b5#explorer/6a2cb4146dbfcbf5f3f6fe1

ORGANIZATION Invernadero_G8_P1 PROJECT PL_G8_ARQUII

Data Explorer

My Queries Data Modeling

CLUSTERS (1)

Search clusters

Grupo8

- PL_G8_ARQUII
- admin
- Invernadero_db
- local

Grupo8

View monitoring Visualize your data Create database Refresh

Database name	Storage size	Data size	Collections	Indexes
PL_G8_ARQUII	303.10 kB	327.51 kB	5	5
admin	0 B	0 B	0	0
Invernadero_db	69.63 kB	105.02 kB	1	1
local	-	-	0	-

Alert System Status: All Good

©2026 MongoDB, Inc. Status Terms Privacy Atlas Blog Contact Sales

Chat

```
ASM modulo_1_medias.s
ASM modulo_2_varianza.s
ASM modulo_3_anomalias.s
ASM modulo_4_prediccion.s
ASM modulo_5_tendencia.s
ASM utils.s
```

